



THE SUPERIOR UNIVERSITY LAHORE

Assignment - 1

Semester: 3rd

Section: All

Session: Fall 2025

Faculty of Computer Science and Information Technology

Subject: Data Structures and Algorithms

QCH:

Total Marks: 20

Name.....

Roll No.....

Instructions:

Write the solution of the given question in hard form and submit the Scan PDF to LMS before the deadline.

Task #	CLO #	Domain and BT Level	Total Points
1	1	C6	10
2	2	C3	10

Task 1: Integrated Document Management System with Undo and Sorting

(10 Marks)

You are developing a document management system for a publishing company that handles document revisions and prioritizes printing tasks. The system uses a stack to manage revision history (for undo functionality) and a queue to schedule print jobs, with priorities sorted using Bubble Sort.

Implement a **stack** to store document revisions. Each revision is a string (e.g., "Version 1: Initial draft", "Version 2: Added chapter").

- ✚ Support push to add a new revision.
- ✚ Support pop to undo the latest revision and display the previous state.
- ✚ Support peek to show the current revision without removing it.
- ✚ Include a function to display the full revision history.
- ✚ Perform two undo operations and show the revision history after each.

Implement a **queue** to manage print jobs, where each job has a priority (integer, lower number = higher priority, e.g., [5, 2, 8, 1]).

- ✚ Support enqueue to add a new print job with a priority, and Support dequeue to process the next job.
- ✚ Support peek to show the next job without removing it.
- ✚ Before printing, the system must sort the print jobs in ascending order of priority using Bubble Sort with the swapped flag optimization, and print the queue state after each pass
- ✚ Enqueue the sorted priorities back into the queue.
- ✚ Sort the print jobs, show the queue after each Bubble Sort pass, and dequeue all jobs to display the final order.

Task 2: Event Planning System with Agenda Management and VIP Admissions

(10 Marks)

You are developing an event planning system for a conference organizer that handles session agendas and prioritizes attendee admissions. The system uses a stack to manage agenda changes (for cancellation functionality) and a queue to schedule admissions, with VIP levels sorted using Bubble Sort. Implement the system in Python using for both stack and queue. Handle edge cases, such as empty stacks or queues, by printing appropriate messages (e.g., "No sessions to cancel" or "No attendees waiting").

Implement a **stack** for Session Agendas, where each session is a string (e.g., "Session 1: Keynote").

- ✚ Support push to add a new session.
- ✚ Support pop to cancel the latest session and return the previous agenda state (print "No sessions to cancel" if empty).
- ✚ Support peek to show the next session without removing it (print "Agenda is empty" if empty).
- ✚ Support is_empty to check if the agenda stack is empty.
- ✚ Include a function to display the full agenda.
- ✚ Perform two cancellation operations (pops) and show the full agenda after each. After the second pop, also check and print if the stack is empty using is_empty.

Implement a **queue** for Attendee Admissions, where each attendee is a tuple of (**attendee name, VIP level**) (e.g., ("Alice", 3)), with higher VIP level numbers indicating higher priority.

- ✚ Support enqueue to add a new attendee.
- ✚ Support dequeue to admit the next attendee (print "No attendees waiting" if empty).
- ✚ Support peek to show the next attendee without removing it (print "No attendees waiting" if empty).
- ✚ Support size to return the number of attendees currently in the queue.
- ✚ Before admitting, dequeue all attendees into a list, sort the list in descending order of VIP level using Bubble Sort with the swapped flag optimization (modify the comparison for descending: swap if $arr[j] < arr[j+1]$), and print the list state after each pass. Enqueue the sorted attendees back into the queue.
- ✚ Sort the attendees, show the queue after each Bubble Sort pass, and dequeue all attendees to display the final admission order. After each dequeue, print the current queue size using size.

Best of Luck Students!